

Comprador

Propiedad	Tipo	Visibilidad	Propósito
nombre	String	private	Guardar el nombre del comprador
email	String private	Guardar el correo del comprador	
cantidadBoletos	int	private	Cantidad de boletos que se desea comprar
presupuesto	double	private	Presupuesto maximo disponible

Método	Retorna	Parámetros	Propósito
Comprador(...)		nombre, email, cantidad, presupuesto	Constructor, set inicial
Getters	Varios		Leer atributos
Setters		Según atributo	Modificar atributos, según sea necesario

Localidad

Propiedad	Tipo	Visibilidad	Propósito
numero	int	private	Identifica la localidad
precio	double	private	Precio del boleto
capacidad	int	private	Capacidad máxima
vendidos	int	private	Boletos vendidos

Método	Retorna	Parámetros	Propósito
hayEspacio()	boolean		Verifica si quedan boletos disponibles
venderBoletos()		cantidad	Registra la venta de boletos
disponibles()	int		Calcula cuántos boletos quedan

Ticket

Propiedad	Tipo	Visibilidad	Propósito
numero	int	private	Número del ticket generado aleatoriamente

Método	Retorna	Parámetros	Propósito
generarTicket()	int		Genera un número aleatorio
esGanador()	boolean	a,b	Verifica si el ticket está entre los números generados

SistemaVentas

Propiedad	Tipo	Visibilidad	Propósito
compradorActual	Comprador	private	Comprador que participa actualmente.
localidades	Localidad[], array	private	Almacena las tres localidades disponibles

Método	Retorna	Parámetros	Propósito
nuevoComprador()			Registra un comprador nuevo
nuevaSolicitud()			Procesa la solicitud

			de compra
disponibilidadTotal()			Muestra boletos vendidos y disponibles
disponibilidadLocalidad()			Consulta una localidad especifica
reporteCaja()	double		Calcula el dinero recaudado

6.) Clasificación de clases

Clase	Clasificación	Justificación
Comprador	Model	Almacena la información del comprador
Localidad	Model	Representa una localidad y sus datos
Ticket	Model	Representa el ticker y su información
SistemaVentas	Controller	Contiene la lógica del proceso de compra y coordina las operaciones
Vista	View	Se encarga de mostrar el menú y recibir datos del usuario
Main	Controller	Crea los objetos e inicia la ejecución del sistema

7. ¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

R// Permite una mejor organización del código, un buen manejo de roles y distinción en lo que se está trabajando. Evita cambiar la lógica en todo el programa, sino que unicamente será necesario hacer la modificación en el controller o model, esto sin necesidad de modificar la interfaz o las demás clases.